

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ**

Кафедра Систем информатики

Направление подготовки 09.06.01 – Информатика и вычислительная техника

ОТЧЕТ

Обучающегося Якутиной Светланы Александровны группы № 22215 курса 4
(Ф.И.О. полностью)

Тема задания: Разработка хранилища корпуса текстов

Новосибирск 2025

Оглавление

Введение	3
Реализация моделей данных.....	4
Реализация REST API для работы с корпусами и текстами	6
Заключение.....	8

Введение

Целью данной лабораторной работы является разработка хранилища корпуса текстов для дальнейшего использования в задачах компьютерной лингвистики в течение семестра.

Для достижения поставленной цели были сформулированы следующие задачи:

- разработка структуры базы данных для хранения корпусов и текстов на основе фреймворка Django;
- реализация моделей данных «Корпус» и «Текст» с использованием ORM Django;
- разработка REST API для создания, редактирования, получения и удаления корпусов и текстов;
- интеграция результатов предыдущих лабораторных работ;
- добавление онтологического слоя с использованием графовой базы данных Neo4j;

Реализация моделей данных

Для хранения данных были реализованы две основные модели: **Corpus** и **Text**.

Модель **Corpus** предназначена для описания корпуса текстов и содержит следующие поля:

- name — название корпуса;
- description — описание корпуса;
- genre — жанр или предметная область.

Модель **Text** предназначена для хранения отдельных текстов и включает:

- name — название текста;
- description — краткое описание;
- content — полный текст;
- corpus — связь с корпусом;
- has_translation — связь с другим текстом.

Пример фрагмента реализации модели текста приведён на листинге 1.

```

class Text(models.Model):
    new *
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    content = models.TextField()
    corpus = models.ForeignKey(
        Corpus,
        related_name='texts',
        on_delete=models.CASCADE
    )
    has_translation = models.ForeignKey(
        to: 'self',
        null=True,
        blank=True,
        on_delete=models.SET_NULL
    )

    def __str__(self):
        new *
        return self.name

```

Листинг 1 – Фрагмент кода модели Text

Реализация REST API для работы с корпусами и текстами

Для взаимодействия с хранилищем были реализованы REST-эндпоинты.

Для **корпусов** реализованы следующие операции:

- создание корпуса (create_corpus);
- получение корпуса вместе со всеми текстами (get_corpus);
- обновление корпуса (update_corpus);
- удаление корпуса (delete_corpus).

Для **текстов** реализованы аналогичные операции:

- создание текста (create_text);
- получение текста (get_text);
- обновление текста (update_text);
- удаление текста (delete_text).

Работа с онтологией

В рамках расширения функциональности проекта были также реализованы эндпоинты для работы с онтологией, хранящейся в графовой базе данных Neo4j.

Для доступа к онтологии используется слой репозитория DriverRepository и OntologyRepository.

Были реализованы REST-эндпоинты для:

- получения полной онтологии и корневых классов;
- получения информации о классах онтологии, их родителях, потомках и объектах;
- создания, обновления и удаления классов;
- добавления и удаления атрибутов классов;

- работы с объектами онтологии (создание, обновление, получение и удаление);
- получения сигнатуры класса.

Пример реализации эндпоинта для получения текста приведён на листинге 2.

```
@api_view(['GET']) 2 usages new *
@permission_classes((AllowAny,))
def get_text(request):
    text_id = request.GET.get('id')

    if not text_id:
        return Response(
            {"error": "Missing 'id' parameter"},
            status=status.HTTP_400_BAD_REQUEST
        )

    try:
        text = text_repo.get(text_id)
        return Response(text, status=status.HTTP_200_OK)
    except Text.DoesNotExist:
        return Response(
            {"error": "Text not found"},
            status=status.HTTP_404_NOT_FOUND
        )
```

Листинг 2 – Эндпоинт получения текста

Заключение

В ходе выполнения лабораторной работы были реализованы все поставленные задачи. Было разработано хранилище корпуса текстов с использованием Django, реализованы модели данных, REST API для управления корпусами и текстами, а также выполнена интеграция с графовой базой данных Neo4j для хранения онтологии предметной области.